CPU time when processing certain inputs, leading to an attack known as a *regular expression denial of service* (ReDoS) attack (see sidebar).

ReDoS Attacks

A *regular expression denial of service* ***(or ReDoS) attack*** occurs when a regular expression can be forced to take a very long time to match a carefully chosen input string. This can happen if the regular expression implementation can be forced to back-track many times to consider different possible ways the expression might match.

As an example, the regular expression `^(a|aa)+$` can match a long string of `a` characters using a repetition of either of the two branches. Given the input string “aaaaaaaaaaaaab” it might first try matching a long sequence of single `a` characters, then when that fails (when it sees the `b` at the end) it will try matching a sequence of single a characters followed by a double-a (`aa`) sequence, then two double-a sequences, then three, and so on. After it has tried all those it might try interleaving single-a and double-a sequences, and so on. There are a lot of ways to match this input, and so the pattern matcher may take a very long time before it gives up. Some regular expression implementations are smart enough to avoid these problems, but many popular programming languages (including Java) are not.[3]

Design your regular expressions so that there is always only a single way to match any input. In any repeated part of the pattern, each input string should only match one of the alternatives. If you’re not sure, prefer using simpler string operations instead.

If you compile and run this new version of the API, you’ll find that you still get a 500 error, but at least you are not sending invalid requests to the database any more. To communicate a more descriptive error back to the user, you can install a Spark exception handler in your `Main` class, as shown in listing 2.9. Go back to the Main.java file in your editor and navigate to the end of the main method. Spark exception handlers are registered by calling the `Spark.exception()` method, which we have already statically imported. The method takes two arguments: the exception class to handle, and then a handler function that will take the exception, the request, and the response objects. The